Guia Prático de Recursion – Ciência da Computação

1️⃣ Conceito Geral

Recursion (Recursão) é quando uma função chama a si mesma para resolver um problema. É uma maneira de quebrar problemas grandes em problemas menores, até chegar a um caso simples que pode ser resolvido diretamente (chamado de caso base).

Pense na recursão como aqueles jogos de plataforma dos anos 90 onde cada fase é igual, mas em menor escala, até você alcançar o final.

2️⃣ Estrutura de uma Função Recursiva

1. Caso base: define quando parar a recursão.
2. Chamada recursiva: a função se chama com dados menores ou modificados.

```
def recursiva(parametro):
  if caso_base(parametro):
    return resultado_simples
  else:
    return recursiva(parametro_modificado)
```

3️⃣ Exemplos Práticos

a) Fatorial

```
def fatorial(n):
  if n == 0:
    return 1  # caso base
  else:
    return n * fatorial(n-1)  # chamada recursiva


print(fatorial(5))  # 120
```

Analogia anos 90: cada inimigo que você derrota gera 1 inimigo menor para derrotar até que não reste nenhum.

b) Fibonacci

```
def fibonacci(n):
  if n <= 1:
    return n  # caso base
  else:
    return fibonacci(n-1) + fibonacci(n-2)  # chamada recursiva


print(fibonacci(6))  # 8
```

Analogia anos 90: cada fase gera duas fases menores que você precisa completar antes de continuar.

c) Caminhar em labirintos

```
def labirinto(posicao, destino, visitados=set()):
  if posicao == destino:
    return True  # caso base
  visitados.add(posicao)
  for proximo in vizinhos(posicao):
    if proximo not in visitados:
      if labirinto(proximo, destino, visitados):
        return True
  return False
```

Analogia: explorar caminhos em um labirinto, voltar quando chega em um beco sem saída.

4️⃣ **Dicas do professor anos 90**

- **Sempre defina um caso base para não entrar em loop infinito.**
- **Ideal para problemas que podem ser divididos em subproblemas iguais.**

- **Pode ser menos eficiente que loops se usado sem cuidado.**
- **Visualize cada chamada como uma fita de jogo que você precisa terminar antes de passar para a anterior.**

5⃣ Comparação Visual (anos 90 style)

Função fatorial(3):

3 * fatorial(2)

2 * fatorial(1)

1 (caso base)

Resultado final: 6

Fila de chamadas Fibonacci(4):

fib(4)

fib(3)

fib(2)

fib(1)

fib(0)

fib(1)

fib(2)

fib(1)

fib(0)